

Tests for variable importance estimation

Stacy Cyrille

2026-06-15

Table of contents

1	Scenario 1: Additive hazards	2
1.1	Objectives	2
1.2	Data generation	2
1.3	Variable importance	13
1.3.1	Global survival stacking	14
1.3.2	Aalen model	17
1.3.3	Survival Super Learner	19
2	Scenario 2: Mixed additive and multiplicative hazards	21
2.1	Objectives	21
2.2	Data generation	22
2.3	Variable importance	34
2.3.1	Global survival stacking	35
2.3.2	Cox-Aalen model	38
2.3.3	Survival Super Learner	41

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(timereg)
library(survML)
library(survivalSL)
library(SuperLearner)
library(randomForestSRC)
library(survex)

source(here::here("R", "utils.R"))
```

```

source(here::here("R", "generate_data.R"))
source(here::here("R", "pred_generators.R"))

# Set seed for reproducibility
set.seed(2026)
# n <- 200000
n <- 1000
V <- 5
sample_split <- TRUE
vims <- c("AUC", "brier")
R <- 100
# R <- 1000

```

1 Scenario 1: Additive hazards

1.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under additive hazards: bias, confidence interval coverage, confidence interval width, rejection probability
- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Aalen model (`timereg::aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

1.2 Data generation

Four binary covariates were generated *independently* with prevalences spanning rare to frequent alterations, $X_j \sim \text{Bernoulli}(0.4)$.

Events times were simulated under the following the additive hazards model:

$$\lambda(t|x) = \lambda_0(t) + \alpha_1(t)X_1 + \alpha_2(t)X_2 + \alpha_3(t)X_3 + \alpha_4(t)X_4$$

with $\lambda_0(t) = 0.05$ (constant) and $\alpha_j(t) = (0.1, 0.05, 0.005, 0)$.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```

p <- 4
sigma <- diag(x = 1, nrow = p, ncol = p)
prev <- rep(0.4, p)
alpha <- c(0.1, 0.05, 0.005, 0)
lambda0 <- 0.05
target_cens <- 0.2
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                                alpha        = alpha,
                                prev         = prev,
                                sigma        = sigma,
                                target_cens  = target_cens,
                                seed         = 2026)

c_max

```

```
[1] 56.65854
```

```

data_scen1 <- simulate_aalen_data(n      = n ,
                                prev     = prev,
                                sigma     = sigma,
                                lambda0   = lambda0,
                                alpha     = alpha,
                                c_max     = c_max)

head(data_scen1)

```

	time	event	X1	X2	X3	X4
1	4.545974	1	1	0	0	1
2	1.926404	1	1	0	1	1
3	5.562858	1	0	1	0	1
4	1.455443	1	1	1	1	0
5	8.037586	0	0	0	0	1
6	23.579391	1	0	0	0	0

```

# Check prevalences
round(colMeans(data_scen1 %>% dplyr::select(starts_with("X"))), 3)

```

	X1	X2	X3	X4
	0.404	0.383	0.424	0.399

```
# Check censoring rate
mean(1 - data_scen1$event)
```

```
[1] 0.2
```

```
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_scen1)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance p-value $H_0: B(t)=0$	
(Intercept)	8.49	0.000
X1	8.82	0.000
X2	7.20	0.000
X3	1.84	0.759
X4	1.81	0.750

Test for time invariant effects

	Kolmogorov-Smirnov test p-value H_0 :constant effect	
(Intercept)	0.557	0.000
X1	1.400	0.000
X2	0.759	0.025
X3	0.368	0.082
X4	0.404	0.054

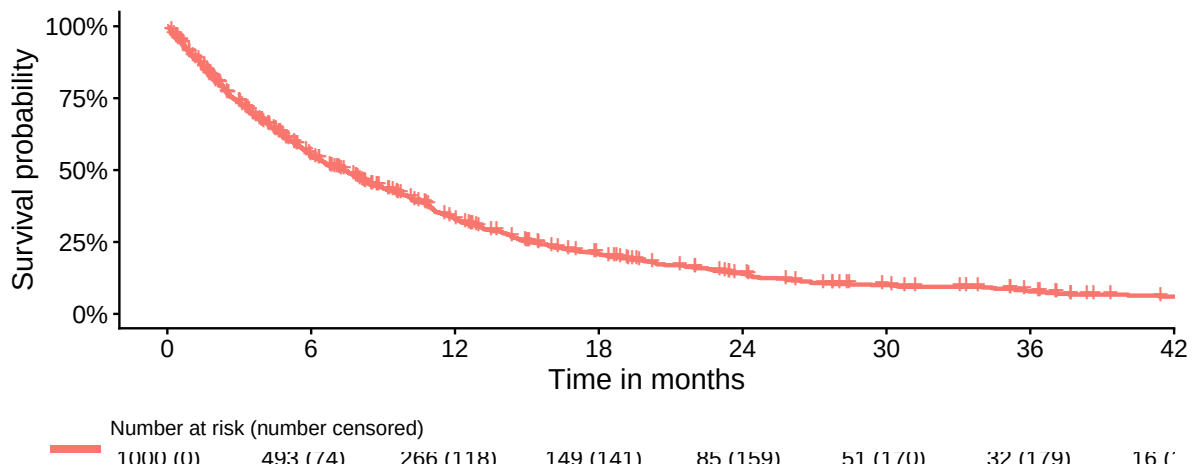
	Cramer von Mises test p-value H_0 :constant effect	
(Intercept)	2.71	0.001
X1	24.00	0.000
X2	6.86	0.013
X3	1.35	0.062
X4	0.85	0.227

Call:
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,
data = data_scen1)

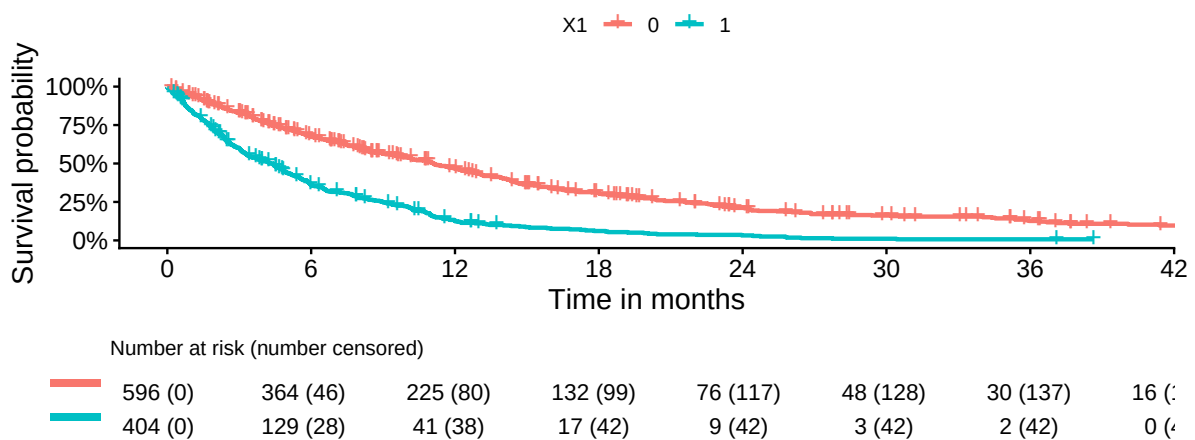
Kaplan-Meier

```
list_form <- c("1",paste0("X",1:p))%>%
  purrr::map(~as.formula(paste("Surv(time,event)~", .x)))
plot_surv(formula = list_form,
  data = data_scen1,
  legend.title = c("", paste0("X",1:p)))
```

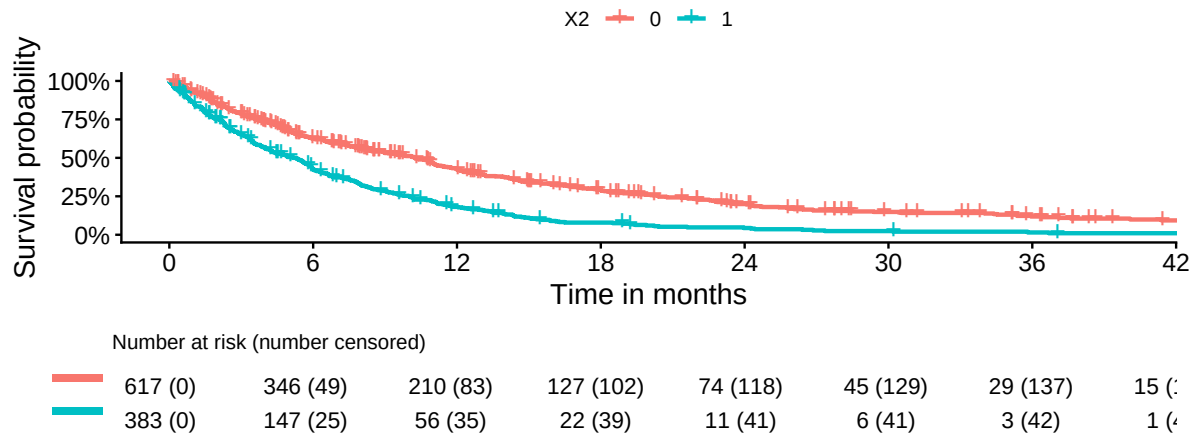
\$data_null_model



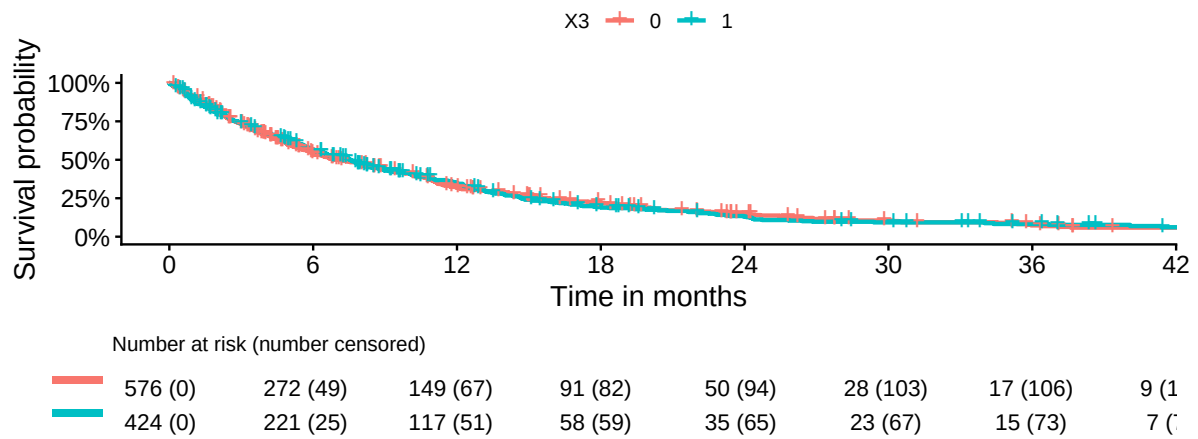
\$data_X1



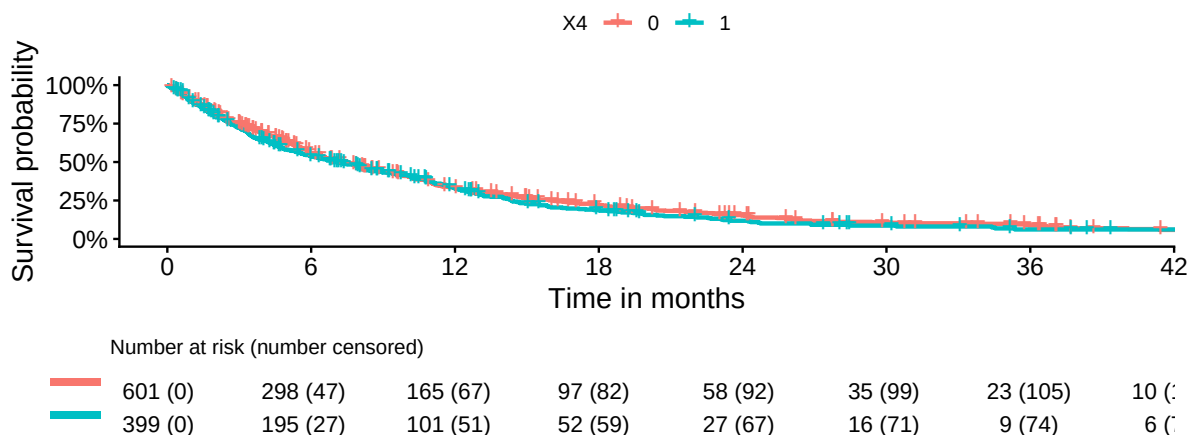
\$data_X2



\$data_X3



\$data_X4



Aalen model

```
# Fit the model
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_scen1)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	8.49	0.000	
X1	8.82	0.000	
X2	7.20	0.000	
X3	1.84	0.740	
X4	1.81	0.754	

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ :constant effect
(Intercept)	0.557	0.000	
X1	1.400	0.000	
X2	0.759	0.020	
X3	0.368	0.080	
X4	0.404	0.065	

	Cramer von Mises test	p-value	H ₀ :constant effect
(Intercept)	2.71	0.000	

X1	24.00	0.000
X2	6.86	0.006
X3	1.35	0.062
X4	0.85	0.220

```
Call:
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,
  data = data_scen1)
```

Test for non-significant effects

The model estimates the cumulative effects

$$A_j(t) = \int_0^t \alpha_j(u) du.$$

The reported *Supremum test* evaluates $H_0 : A_j(t) = 0$, meaning the covariate has no effect on the hazard over time.

Interpretation of the results

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ Strong evidence that X_2 affects the hazard
- X_3 : $p = 0.7 \rightarrow$ No evidence that X_3 affects the hazard
- X_4 : $p = 0.8 \rightarrow$ No evidence that X_4 affects the hazard

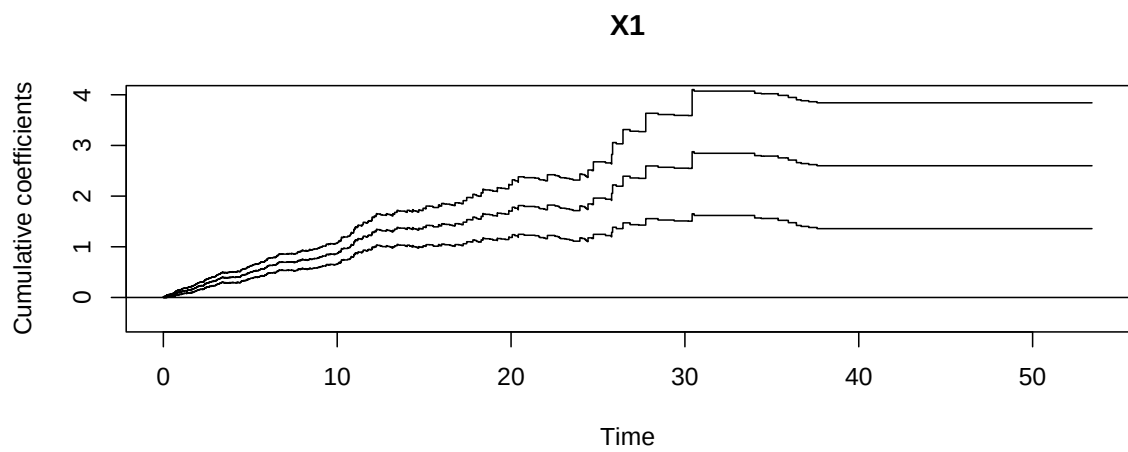
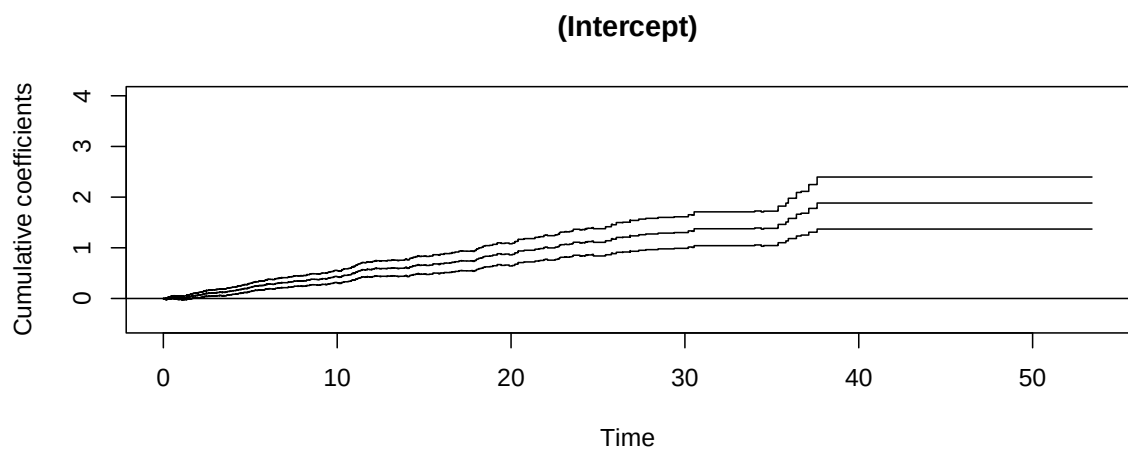
Test for time invariant effects

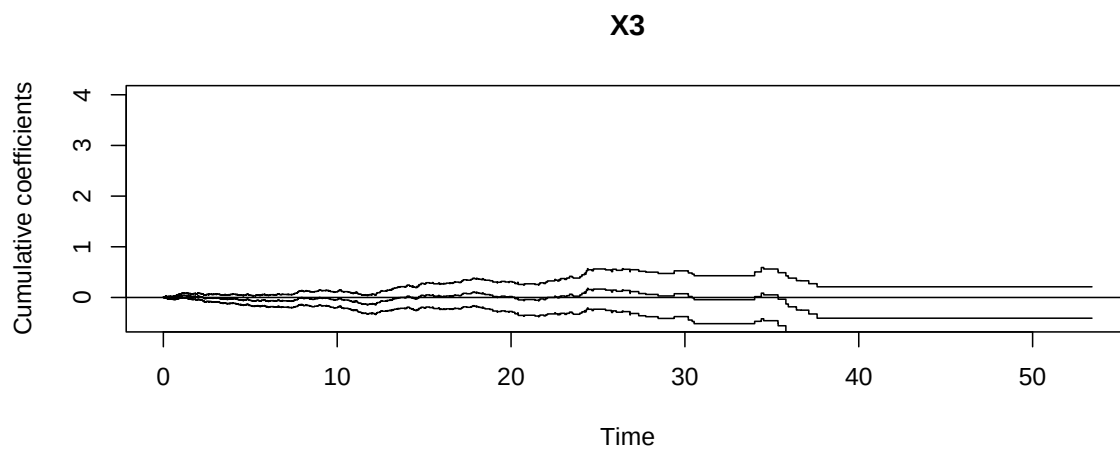
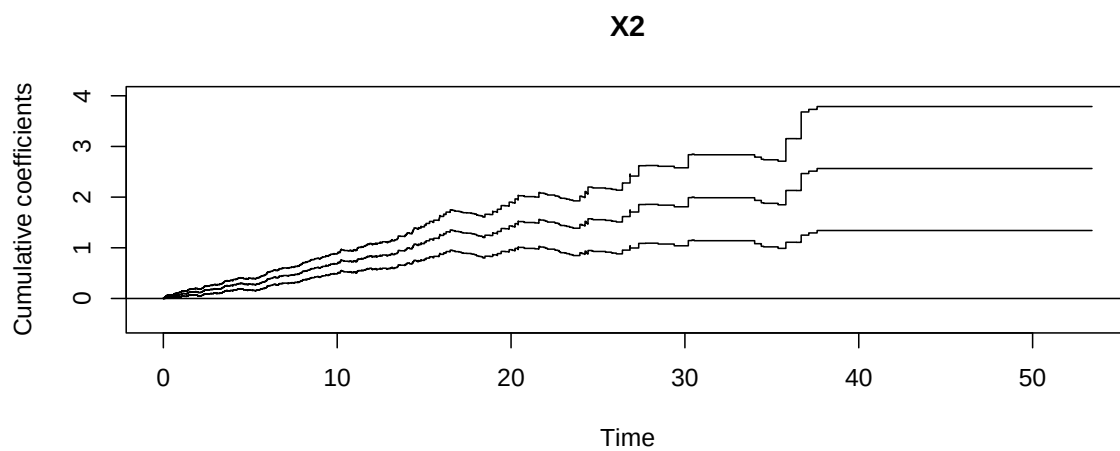
Interpretation of the results of the Kolmogorov–Smirnov test

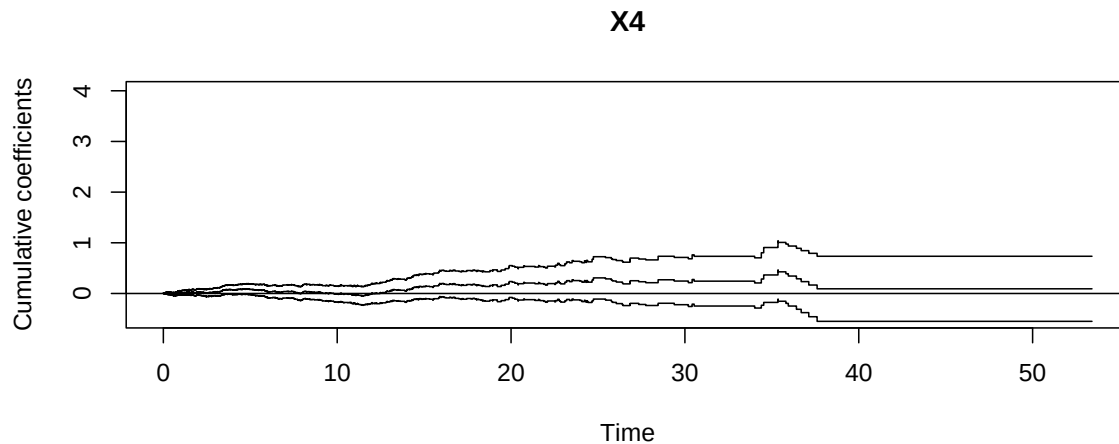
Small p-values *rigtharrow* evidence of time-varying effects.

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that effect of X_1 changes over time
- X_2 : $p = 0.020 \rightarrow$ Strong evidence that effect of X_2 changes over time
- X_3 : $p = 0.080 \rightarrow$ No strong evidence against constant effect
- X_4 : $p = 0.065 \rightarrow$ No evidence of time variation

```
# Plot the non-parametric cumulative regression coefficients with 95% CI
plot(aalen_fit, ylim = c(-0.5,4))
```





We simulated $R = 1000$ datasets.

```
test_aalen <- replicate(R, {
  # Generate data
  data <- simulate_aalen_data(n, prev, sigma, lambda0, alpha, c_max)
  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(starts_with("X"))) %>%
    stats::setNames(paste0("prev_", names(.)))
  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Aalen model
  fit_obj <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data)
  # Tests
  pval_supremum <- fit_obj[["pval.testBeq0"]]
  names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
  pval_KS <- fit_obj[["pval.testBeqC"]]
  names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
  pval_CvM <- fit_obj[["pval.testBeqC.is"]]
  names(pval_CvM) <- paste0("pval_CvM_", names(pval_CvM))

  c(prev, cens_rate, pval_supremum, pval_KS, pval_CvM)
}, simplify = F) %>%
bind_rows
```

```
data.frame(colMeans(test_aalen))%>%
  tibble::rownames_to_column()%>%
  kable(col.names = c("Name", "Value"))
```

Name	Value
prev_X1	0.39820
prev_X2	0.40080
prev_X3	0.40028
prev_X4	0.40156
cens_rate	0.20109
pval_supremum_(Intercept)	0.00000
pval_supremum_X1	0.00000
pval_supremum_X2	0.00005
pval_supremum_X3	0.47247
pval_supremum_X4	0.52200
pval_KS_(Intercept)	0.01088
pval_KS_X1	0.01488
pval_KS_X2	0.03936
pval_KS_X3	0.41978
pval_KS_X4	0.46846
pval_CvM_(Intercept)	0.01313
pval_CvM_X1	0.01764
pval_CvM_X2	0.04580
pval_CvM_X3	0.43348
pval_CvM_X4	0.48313

```
test_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  kable(col.names = c("p-value", "Rejection probability"))
```

p-value	Rejection probability
pval_supremum_(Intercept)	1.00
pval_supremum_X1	1.00
pval_supremum_X2	1.00
pval_supremum_X3	0.05
pval_supremum_X4	0.06

p-value	Rejection probability
pval_KS_(Intercept)	0.97
pval_KS_X1	0.91
pval_KS_X2	0.82
pval_KS_X3	0.06
pval_KS_X4	0.04
pval_CvM_(Intercept)	0.96
pval_CvM_X1	0.93
pval_CvM_X2	0.79
pval_CvM_X3	0.08
pval_CvM_X4	0.03

1.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Aalen additive hazards model (`timereg::aalen`)
- Super Learner for survival prediction (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
time <- data_scen1$time
event <- data_scen1$event
X <- data_scen1 %>% dplyr::select(starts_with("X"))
surv_aalen_fit <- survival::survfit(survival::Surv(time, event)~1)
landmark_times <- quantile(surv_aalen_fit,
                           probs = c(#0.25,
                                     0.75))$quantile
landmark_times
```

```
75
15.08673
```

```
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

```
[1] 681
```

We generated cross-fitting and sample-splitting folds.

```
foldes <- survML::generate_folds(n, V, sample_split)
cf_folds <- foldes$cf_folds
table(cf_folds)
```

```
cf_folds
  1  2  3  4  5  6  7  8  9 10
100 100 100 100 100 100 100 100 100 100
```

```
ss_folds <- foldes$ss_folds
table(ss_folds)
```

```
ss_folds
  0  1
500 500
```

1.3.1 Global survival stacking

```
V0_stackG_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
# str(V0_stackG_preds_scen1)

CV_full_preds <- V0_stackG_preds_scen1$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen1$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen1$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen1$CV_G_preds

indxes <- 1:ncol(X)
vim_stackG_scen1 <- purrr::map_dfr(indxes, function(indx_i) {
```

```

char_indx <- as.character(indx_i)
indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  cf_folds = cf_folds,
  indx = indx,
  full_preds_train = CV_full_preds_train
)

purrr::map_dfr(vims, function(vim) {

  output <- switch(
    vim,
    brier = survML::vim_brier(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = CV_full_preds,
      fs_hat = CV_reduced_preds,
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    ),
    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )
})

```

```

    )
  )

  output %>%
    dplyr::mutate(
      vim = vim,
      indx = char_indx
    )
})
})

kable(vim_stackG_scen1 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large AUC	small AUC
1	15.087	0.060	49.417	-0.556	0.676	0.425	0.578	0.518
2	15.087	0.073	46.686	-0.526	0.672	0.405	0.578	0.504
3	15.087	0.074	45.854	-0.520	0.667	0.404	0.578	0.504
4	15.087	0.051	50.892	-0.574	0.677	0.436	0.578	0.526

```

kable(vim_stackG_scen1 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large Brier	small Brier
1	15.087	-0.084	5.115	-0.282	0.115	0.796	-1.023	-0.939
2	15.087	-0.086	5.175	-0.286	0.113	0.802	-1.023	-0.936
3	15.087	-0.084	4.931	-0.279	0.111	0.801	-1.023	-0.939
4	15.087	-0.085	4.926	-0.279	0.110	0.804	-1.023	-0.938

1.3.2 Aalen model

```
VO_aalen_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "aalen",
  cf_folds = cf_folds)
# str(VO_aalen_preds_scen1)

CV_full_preds <- VO_aalen_preds_scen1$CV_full_preds
CV_full_preds_train <- VO_aalen_preds_scen1$CV_full_preds_train
CV_S_preds <- VO_aalen_preds_scen1$CV_S_preds
CV_G_preds <- VO_aalen_preds_scen1$CV_G_preds

indx_s <- 1:ncol(X)
vim_aalen_scen1 <- purrr::map_dfr(indx_s, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
```

```

    f_hat = CV_full_preds,
    fs_hat = CV_reduced_preds,
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  ),
  AUC = survML::vim_AUC(
    time = time,
    event = event,
    approx_times = approx_times,
    landmark_times = landmark_times,
    f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
    fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_aalen_scen1 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large AUC	small AUC
1	15.087	0.146	0.696	0.073	0.219	0.000	0.792	0.646
2	15.087	0.122	0.670	0.050	0.193	0.000	0.792	0.671
3	15.087	0.042	0.675	-0.030	0.114	0.125	0.792	0.750
4	15.087	0.050	0.647	-0.021	0.120	0.083	0.792	0.742

```
kable(vim_aalen_scen1 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier"))
```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large Brier	small Brier
1	15.087	-0.088	0.176	-0.125	-0.051	1.000	-0.552	-0.464
2	15.087	-0.080	0.184	-0.117	-0.042	1.000	-0.552	-0.472
3	15.087	-0.053	0.188	-0.091	-0.015	0.997	-0.552	-0.499
4	15.087	-0.054	0.188	-0.092	-0.016	0.997	-0.552	-0.499

1.3.3 Survival Super Learner

```
V0_survivalSL_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "survivalSL",
  cf_folds = cf_folds)
# str(V0_stackG_preds_scen1)

CV_full_preds <- V0_survivalSL_preds_scen1$CV_full_preds
CV_full_preds_train <- V0_survivalSL_preds_scen1$CV_full_preds_train
CV_S_preds <- V0_survivalSL_preds_scen1$CV_S_preds
CV_G_preds <- V0_survivalSL_preds_scen1$CV_G_preds
```

```

indx_s <- 1:ncol(X)
vim_survivalSL_scen1 <- purrr::map_dfr(indx_s, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),
      AUC = survML::vim_AUC(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
        fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,

```

```

      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )

  output %>%
    dplyr::mutate(
      vim = vim,
      indx = char_indx
    )
})
})

kable(vim_survivalSL_scen1 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small AUC"))

kable(vim_survivalSL_scen1 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier"))

```

2 Scenario 2: Mixed additive and multiplicative hazards

2.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under mixed additive and multiplicative hazards: bias, confidence interval coverage, confidence interval width, rejection probability

- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Cox-Aalen model (`timereg::cox.aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

2.2 Data generation

The covariates are generated *independently* as follows:

- $X_1, X_2, X_3 \sim \mathcal{B}(0.4)$
- $Z_1, Z_2 \sim \mathcal{B}(0.4)$
- $Z_3 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1 , X_2 and X_3 with coefficients $\alpha_1 = 0.1$, $\alpha_2 = 0.05$ and $\alpha_3 = 0$, respectively. Multiplicative (proportional hazards) effects were assigned to Z_1 , Z_2 and Z_3 with coefficients $\beta_1 = 1$, $\beta_2 = 0.5$ and $\beta_3 = 0$, corresponding to hazard ratios of approximately 2.72, 1.65 and 1.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
prev_add <- c(0.4, 0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 3, ncol = 3)
alpha <- c(0.1, 0.05, 0)
prev_mult <- c(0.4, 0.4)
sigma_mult_bin <- diag(x = 1, nrow = 2, ncol = 2)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta <- c(1, 0.5, 0)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_cox_aalen(lambda0      = lambda0,
                                   alpha        = alpha,
                                   beta          = beta,
                                   prev_add      = prev_add,
                                   sigma_add     = sigma_add,
                                   prev_mult     = prev_mult,
                                   mean_mult     = mean_mult,
                                   sigma_mult_bin = sigma_mult_bin,
                                   sigma_mult_cont = sigma_mult_cont,
                                   target_cens    = target_cens,
```

```

                                seed = 2026)
c_max

```

```
[1] 151.3418
```

```

data_scen2 <- simulate_cox_aalen_data(
  n, lambda0, alpha, beta,
  prev_add, sigma_add, prev_mult, mean_mult,
  sigma_mult_bin, sigma_mult_cont, c_max
)
head(data_scen2)

```

```

      time event X1 X2 X3 Z1 Z2      Z3
1 15.9040045    1  0  0  0  1  0.06569121
2  0.0432154    1  1  0  0  1 -0.29487658
3  1.6391049    1  0  1  0  0  0.19398223
4 30.5502647    1  1  0  1  0  1.33259324
5  3.1981744    0  0  0  1  1 -1.51629094
6  4.7261423    1  1  0  1  0  0.23890336

```

```

# Check prevalences
colMeans(data_scen2 %>% dplyr::select(X1:X3,Z1:Z2))

```

```

      X1      X2      X3      Z1      Z2
0.383 0.393 0.413 0.408 0.393

```

```

# Check mean and sd
round(mean(data_scen2$Z3), 3)

```

```
[1] 0
```

```
sd(data_scen2$Z3)
```

```
[1] 1
```

```

# Check censoring rate
mean(1 - data_scen2$event)

```

```
[1] 0.055
```

```
cox_aalen_fit <- timereg::cox.aalen(
  Surv(time, event)~X1+X2+X3+prop(Z1)+prop(Z2)+prop(Z3),
  data = data_scen2, n.sim = 1000, basesim = 1
)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

Supremum-test of significance p-value $H_0: B(t)=0$

(Intercept)	10.20	0.000
X1	8.13	0.000
X2	5.67	0.000
X3	1.34	0.941

Test for time invariant effects

Kolmogorov-Smirnov test p-value H_0 :constant effect

(Intercept)	1.0800	0.000
X1	1.6100	0.001
X2	2.4900	0.000
X3	0.0967	0.934

Proportional Cox terms :

	Coef.	SE Robust	SE D2log(L) ⁻¹	z	P-val	lower2.5%
prop(Z1)	1.1200	0.0737	0.0706	0.0733	15.90	0.00e+00
prop(Z2)	0.5140	0.0713	0.0646	0.0695	7.95	1.78e-15
prop(Z3)	0.0276	0.0330	0.0310	0.0329	0.89	3.73e-01

upper97.5%

prop(Z1)	1.2600
prop(Z2)	0.6540
prop(Z3)	0.0923

Test of Proportionality

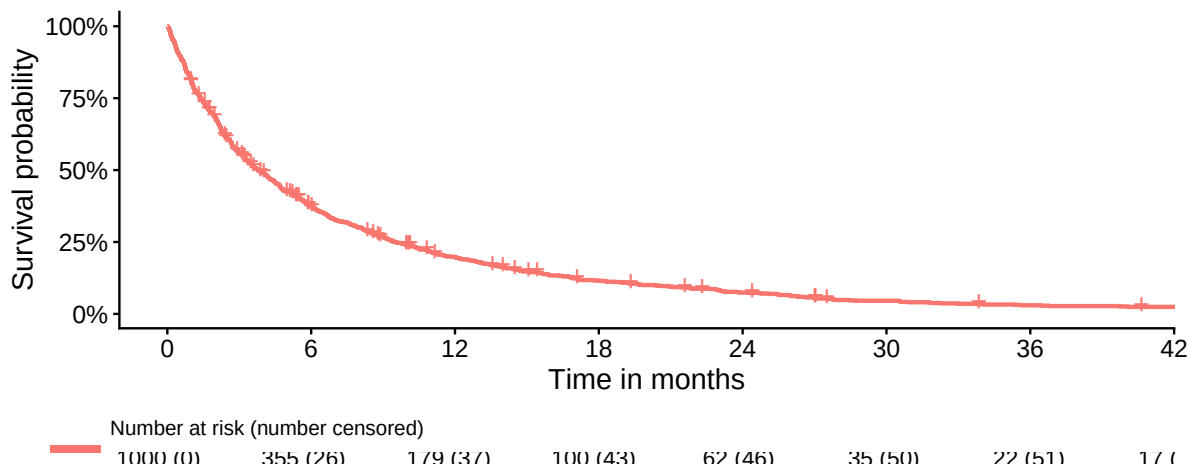
sup| hat U(t) | p-value H_0

prop(Z1)	11.8	0.322
prop(Z2)	15.8	0.130
prop(Z3)	24.2	0.426

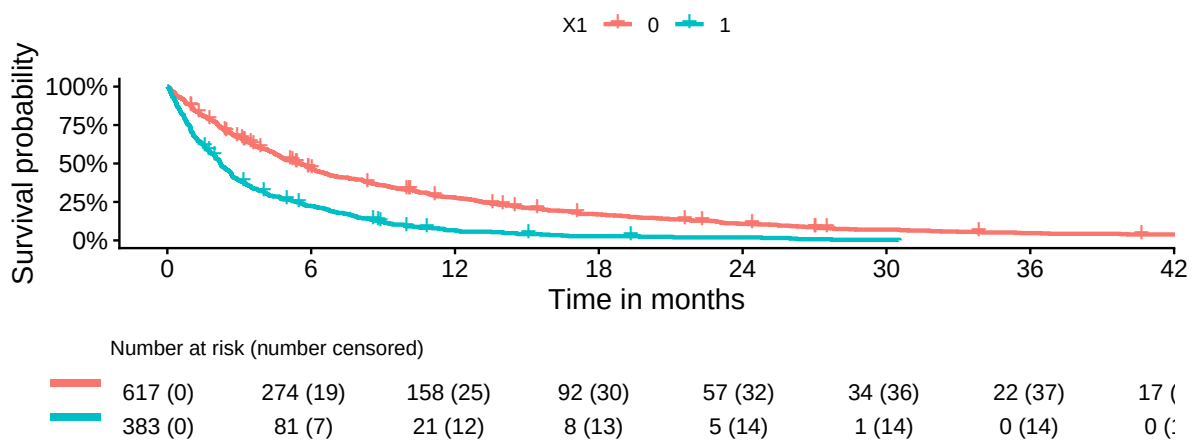
Kaplan-Meier


```
list_form <- c("1",paste0("X",1:3), paste0("Z",1:2))%>%
  purrr::map(~as.formula(paste("Surv(time,event)~", .x)))
plot_surv(formula = list_form,
  data = data_scen2,
  legend.title = c("", paste0("X",1:3), paste0("Z",1:2)))
```

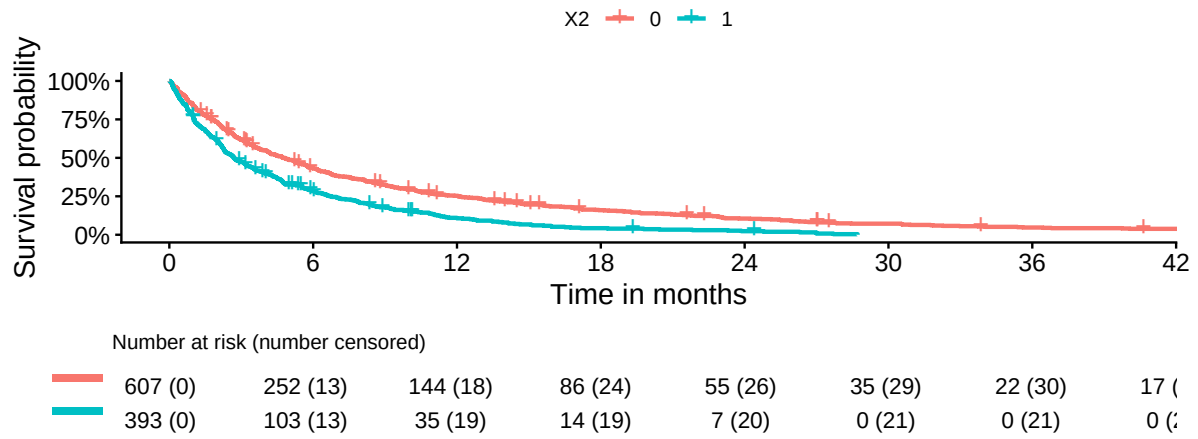
\$data_null_model



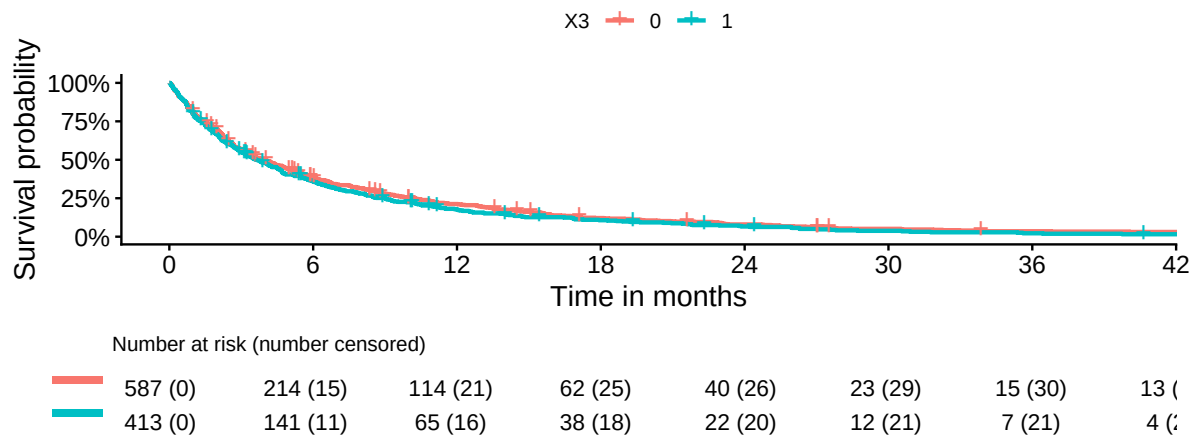
\$data_X1



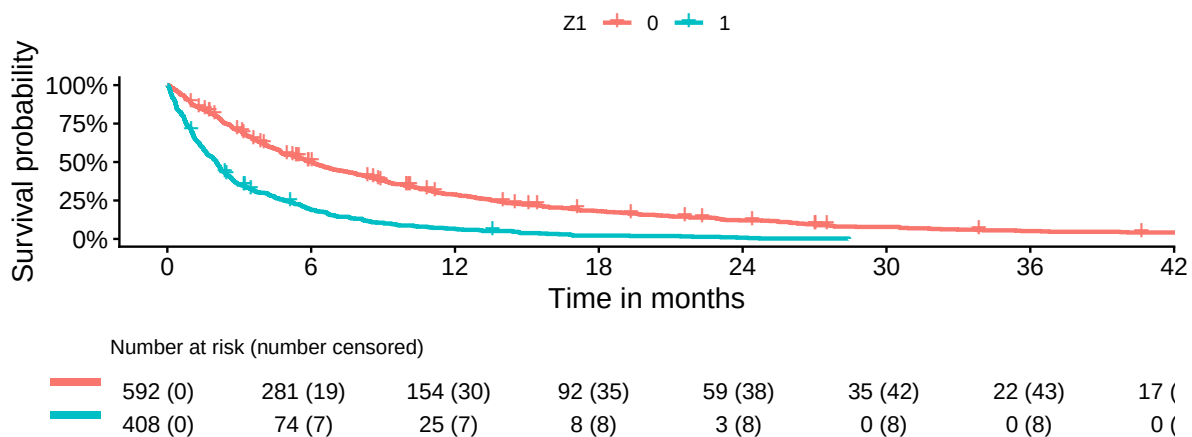
\$data_X2



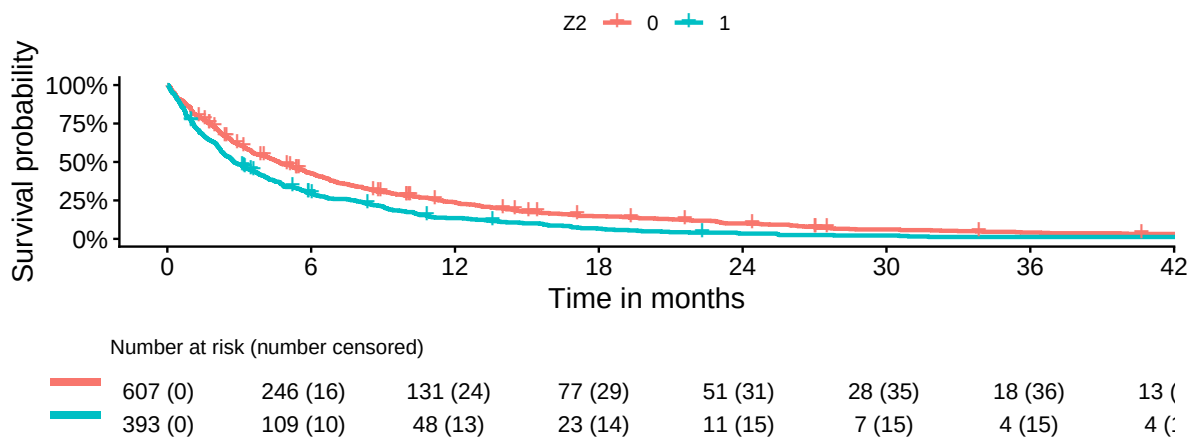
\$data_X3



\$data_Z1



\$data_Z2



Cox-Aalen model

```
# Fit Cox-Aalen model
cox_aalen_fit <- timereg::cox.aalen(
  Surv(time, event)~X1+X2+X3+prop(Z1)+prop(Z2)+prop(Z3),
  data = data_scen2, n.sim = 1000, basesim = 1)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

Supremum-test of significance p-value $H_0: B(t)=0$

(Intercept)	10.20	0.000
X1	8.13	0.000
X2	5.67	0.000
X3	1.34	0.941

Test for time invariant effects

Kolmogorov-Smirnov test p-value H_0 : constant effect

(Intercept)	1.0800	0.000
X1	1.6100	0.001
X2	2.4900	0.000
X3	0.0967	0.941

Proportional Cox terms :

	Coef.	SE Robust	SE	D2log(L) ⁻¹	z	P-val	lower2.5%
prop(Z1)	1.1200	0.0737	0.0706	0.0733	15.90	0.00e+00	0.9760
prop(Z2)	0.5140	0.0713	0.0646	0.0695	7.95	1.78e-15	0.3740
prop(Z3)	0.0276	0.0330	0.0310	0.0329	0.89	3.73e-01	-0.0371

upper97.5%

prop(Z1)	1.2600
prop(Z2)	0.6540
prop(Z3)	0.0923

Test of Proportionality

	sup	hat U(t)	p-value H_0
prop(Z1)	11.8	0.351	
prop(Z2)	15.8	0.110	
prop(Z3)	24.2	0.400	

Test for non-significant effects

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ No evidence that X_2 affects the hazard

Test for time invariant effects

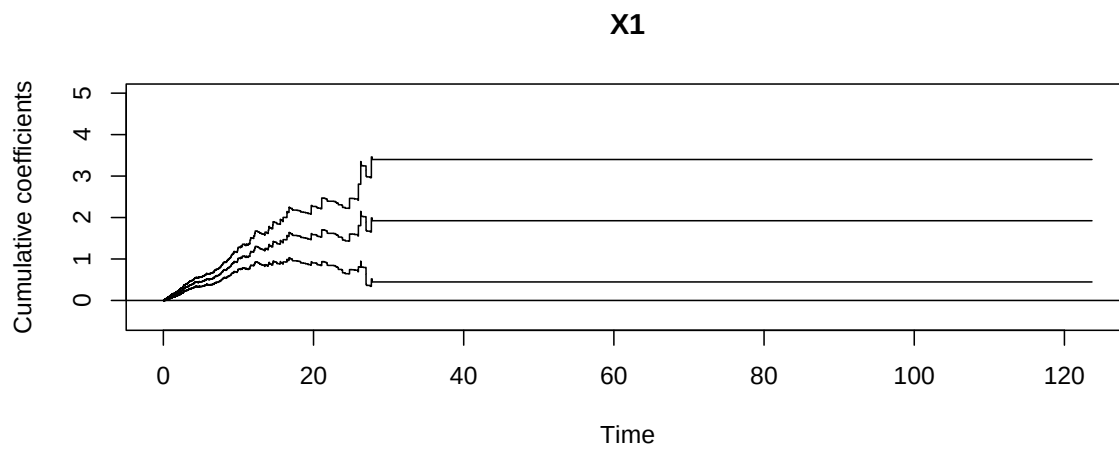
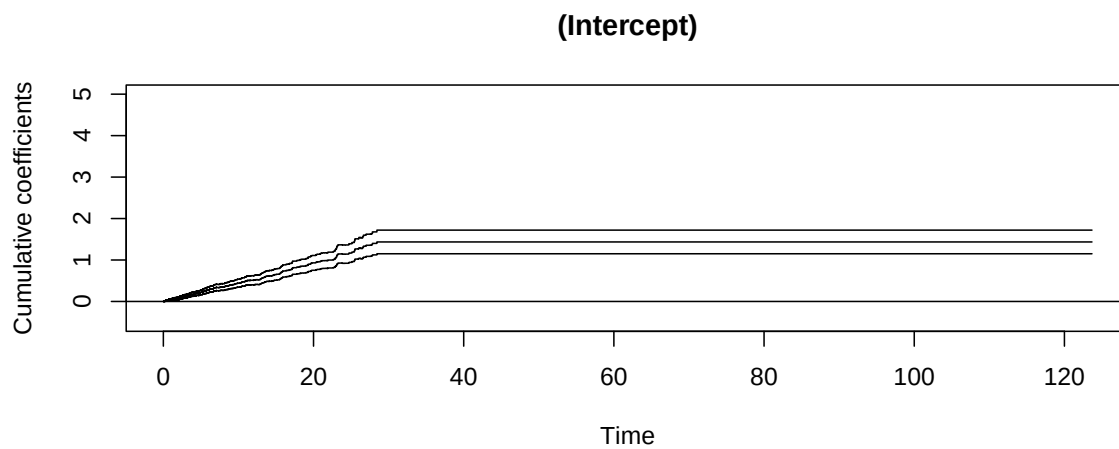
- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time

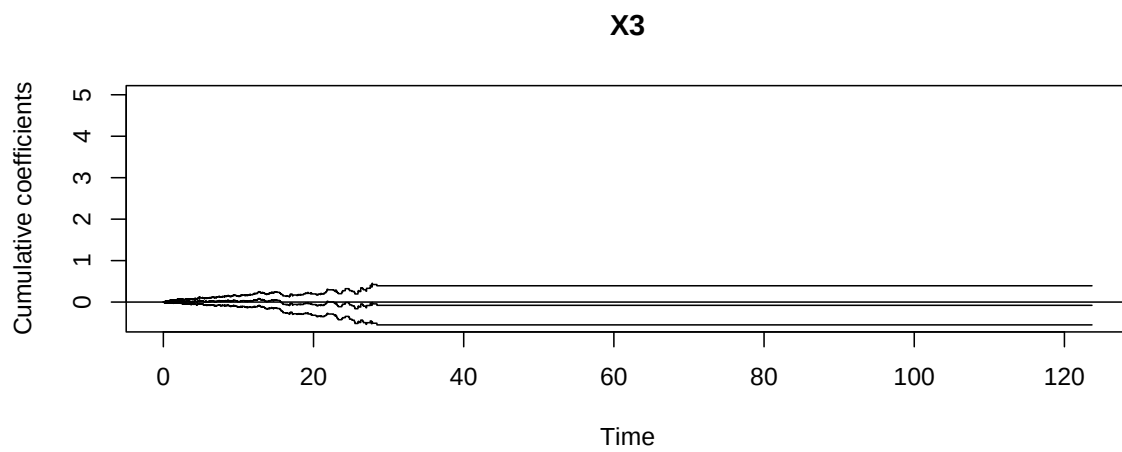
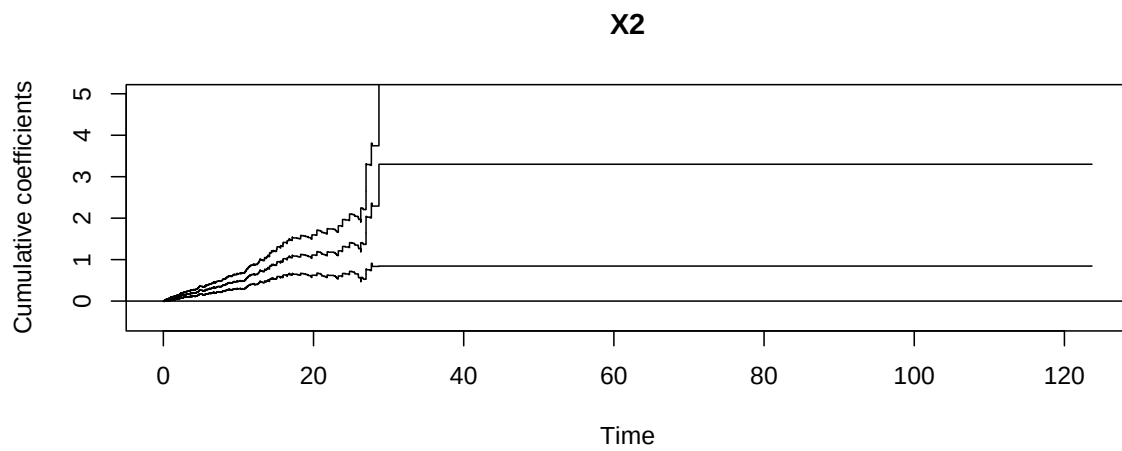
- X_1 : $p=0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p<0.001 \rightarrow$ No evidence of time variation

Test of proportionality

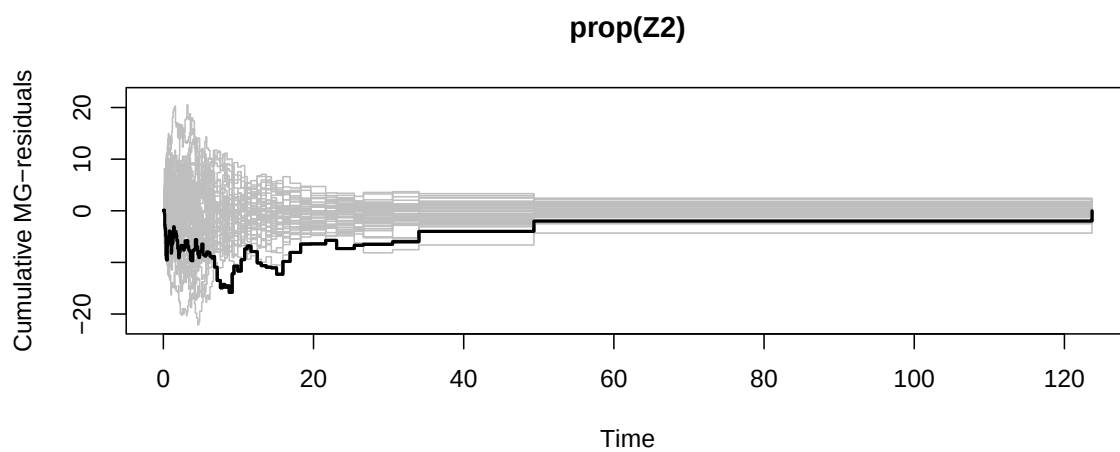
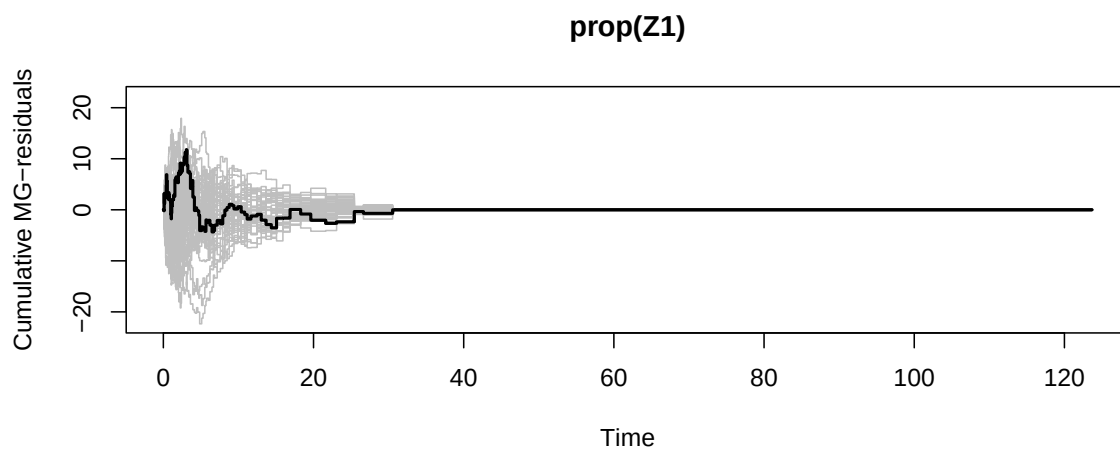
- X_3 : $p=0.4 \rightarrow$ Compatible with a proportional effect
- X_4 : $p=0.11 \rightarrow$ Compatible with a proportional effect

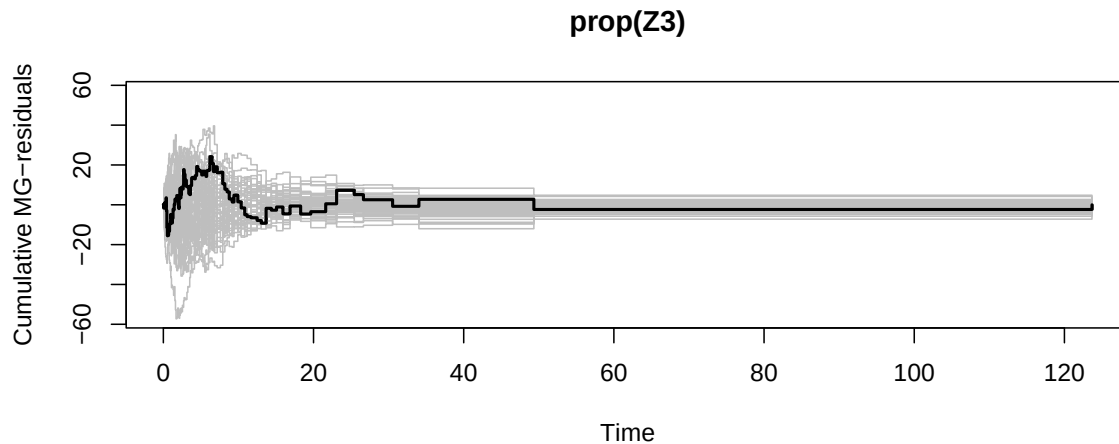
```
# Plot additive Aalen terms
plot(cox_aalen_fit, ylim = c(-0.5,5))
```





```
# Plot multiplicative terms  
plot(cox_aalen_fit, score= TRUE)
```





We simulated $R = 1000$ datasets.

```
test_cox_aalen <- replicate(R, {
  # Generate data
  data <- simulate_cox_aalen_data(n, lambda0, alpha, beta,
                                prev_add, sigma_add, prev_mult, mean_mult,
                                sigma_mult_bin, sigma_mult_cont, c_max)

  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(X1:X3))%>%
    stats::setNames(paste0("prev_", names(.)))
  # Mean and sd Z3
  mean_sd_Z3 <- stats::setNames(c(mean(data$Z3), sd(data$Z3)),
                                c("mean_Z3", "sd_Z3"))

  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Cox-Aalen model
  fit_obj <- timereg::cox.aalen(
    Surv(time, event)~X1+X2+X3+prop(Z1)+prop(Z2)+prop(Z3),
    data = data, n.sim = 1000, basesim = 1
  )
  # Additive terms
  pval_supremum <- fit_obj[["pval.testBeq0"]]
  names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
  pval_KS <- fit_obj[["pval.testBeqC"]]
```



```

names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
# pval_CvM <- fit_obj[["pval.testBeqC.is"]]
# names(pval_CvM) <- paste0("pval_CvM", names(pval_CvM))
# Multiplicative terms
coef <- coef(fit_obj)[,"Coef."]
# Test for proportional
pval_prop <- stats::setNames(
  fit_obj$pval.Prop,
  gsub("prop\\(|\\|\\|)", "", paste0("pval_prop_", names(fit_obj$pval.Prop)))
)

c(prev, mean_sd_Z3, cens_rate,
  pval_supremum, pval_KS, #pval_CvM,
  coef, pval_prop)

}, simplify = F)%>%
bind_rows()

data.frame(round(colMeans(test_cox_aalen),3))%>%
  tibble::rownames_to_column()%>%
  kable(col.names = c("Name", "Value"))

```

Name	Value
prev_X1	0.402
prev_X2	0.399
prev_X3	0.398
mean_Z3	0.000
sd_Z3	1.000
cens_rate	0.051
pval_supremum_(Intercept)	0.000
pval_supremum_X1	0.000
pval_supremum_X2	0.000
pval_supremum_X3	0.447
pval_KS_(Intercept)	0.000
pval_KS_X1	0.001
pval_KS_X2	0.007
pval_KS_X3	0.503
prop(Z1)	1.022
prop(Z2)	0.538
prop(Z3)	-0.002
pval_prop_Z1	0.490

Name	Value
pval_prop_Z2	0.356
pval_prop_Z3	0.519

```
test_cox_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  mutate(value = round(value, 3))%>%
  kable(col.names = c("p-value", "Rejection probability"))
```

p-value	Rejection probability
pval_supremum_(Intercept)	1.00
pval_supremum_X1	1.00
pval_supremum_X2	1.00
pval_supremum_X3	0.02
pval_KS_(Intercept)	1.00
pval_KS_X1	0.99
pval_KS_X2	0.97
pval_KS_X3	0.05
pval_prop_Z1	0.08
pval_prop_Z2	0.09
pval_prop_Z3	0.05

2.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Cox-Aalen model (`timereg::cox.aalen`)
- Survival Super Learner (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```

time <- data_scen2$time
event <- data_scen2$event
X <- data_scen2 %>% dplyr::select(starts_with(c("X","Z")))
surv_cox_aalen_fit <- survival::survfit(survival::Surv(time, event)~1)
landmark_times <- quantile(surv_cox_aalen_fit,
                           probs = c(#0.25,
                                     0.75))$quantile
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)

```

```
[1] 737
```

We generated cross-fitting and sample-splitting folds.

```

folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)

```

```

cf_folds
  1   2   3   4   5   6   7   8   9  10
100 100 100 100 100 100 100 100 100 100

```

```

ss_folds <- folds$ss_folds
table(ss_folds)

```

```

ss_folds
  0   1
500 500

```

2.3.1 Global survival stacking

```

V0_stackG_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,

```

```

nuisance = "stackG",
cf_folds = cf_folds)
# str(V0_stackG_preds_scen2)

CV_full_preds <- V0_stackG_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen2$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen2$CV_G_preds

indx <- 1:ncol(X)
vim_stackG_scen2 <- purrr::map_dfr(indx, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),

```

```

    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )
}

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen2 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large AUC	small AUC
1	9.531	0.026	22.986	-0.394	0.446	0.452	0.808	0.782
2	9.531	0.010	22.456	-0.405	0.425	0.481	0.808	0.798
3	9.531	-0.013	22.535	-0.429	0.403	0.525	0.808	0.821
4	9.531	0.078	29.408	-0.397	0.553	0.374	0.808	0.730
5	9.531	-0.022	24.704	-0.458	0.413	0.540	0.808	0.830
6	9.531	-0.013	22.175	-0.426	0.400	0.524	0.808	0.821

```

kable(vim_stackG_scen2 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier")

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large Brier	small Brier
1	9.531	0.032	0.550	-0.033	0.097	0.168	-0.72	-0.752
2	9.531	0.032	0.549	-0.033	0.097	0.169	-0.72	-0.751
3	9.531	0.033	0.551	-0.032	0.098	0.161	-0.72	-0.753
4	9.531	0.028	0.552	-0.037	0.094	0.196	-0.72	-0.748
5	9.531	0.032	0.549	-0.033	0.097	0.167	-0.72	-0.752
6	9.531	0.033	0.550	-0.032	0.098	0.163	-0.72	-0.752

2.3.2 Cox-Aalen model

```

V0_cox_aalen_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "cox.aalen",
  cf_folds = cf_folds
)
# str(V0_cox_aalen_preds_scen2)

CV_full_preds <- V0_cox_aalen_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_cox_aalen_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_cox_aalen_preds_scen2$CV_S_preds
CV_G_preds <- V0_cox_aalen_preds_scen2$CV_G_preds

indx <- 1:ncol(X)
vim_cox_aalen_scen2 <- purrr::map_dfr(indxs, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

```

```

CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  cf_folds = cf_folds,
  indx = indx,
  full_preds_train = CV_full_preds_train
)

purrr::map_dfr(vims, function(vim) {

  output <- switch(
    vim,
    brier = survML::vim_brier(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = CV_full_preds,
      fs_hat = CV_reduced_preds,
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    ),
    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )
})

```

```

    output %>%
      dplyr::mutate(
        vim = vim,
        indx = char_indx
      )
  })
})

kable(vim_cox_aalen_scen2 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large AUC	small AUC
1	9.531	0.081	0.497	0.020	0.143	0.005	0.817	0.736
2	9.531	0.069	0.469	0.009	0.129	0.012	0.817	0.748
3	9.531	0.027	0.433	-0.030	0.085	0.176	0.817	0.790
4	9.531	0.093	0.534	0.029	0.157	0.002	0.817	0.724
5	9.531	0.045	0.455	-0.014	0.104	0.067	0.817	0.772
6	9.531	0.027	0.415	-0.029	0.084	0.174	0.817	0.790

```

kable(vim_cox_aalen_scen2 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small

```

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large Brier	small Brier
1	9.531	-0.049	0.186	-0.087	-0.012	0.995	-0.565	-0.516
2	9.531	-0.033	0.196	-0.072	0.006	0.954	-0.565	-0.532
3	9.531	-0.017	0.194	-0.056	0.022	0.805	-0.565	-0.548
4	9.531	-0.057	0.182	-0.094	-0.019	0.999	-0.565	-0.508
5	9.531	-0.022	0.193	-0.060	0.017	0.868	-0.565	-0.543

Variable	Horizon	VIM	var(VIM)	CIL	CIU	p	large Brier	small Brier
6	9.531	-0.022	0.191	-0.060	0.016	0.869	-0.565	-0.543

2.3.3 Survival Super Learner

```

VO_survivalSL_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "survivalSL",
  cf_folds = cf_folds)
# str(VO_stackG_preds_scen2)

CV_full_preds <- VO_survivalSL_preds_scen2$CV_full_preds
CV_full_preds_train <- VO_survivalSL_preds_scen2$CV_full_preds_train
CV_S_preds <- VO_survivalSL_preds_scen2$CV_S_preds
CV_G_preds <- VO_survivalSL_preds_scen2$CV_G_preds

indx <- 1:ncol(X)
vim_survivalSL_scen2 <- purrr::map_dfr(indx, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,

```

```

brier = survML::vim_brier(
  time = time,
  event = event,
  approx_times = approx_times,
  landmark_times = landmark_times,
  f_hat = CV_full_preds,
  fs_hat = CV_reduced_preds,
  S_hat = CV_S_preds,
  G_hat = CV_G_preds,
  cf_folds = cf_folds,
  sample_split = sample_split,
  ss_folds = ss_folds
),
AUC = survML::vim_AUC(
  time = time,
  event = event,
  approx_times = approx_times,
  landmark_times = landmark_times,
  f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
  fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
  S_hat = CV_S_preds,
  G_hat = CV_G_preds,
  cf_folds = cf_folds,
  sample_split = sample_split,
  ss_folds = ss_folds
)
)
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_survivalSL_scen2 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),

```

```

      col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small
kable(vim_survivalSL_scen2 %>%
      dplyr::filter(vim == "brier")%>%
      tibble::remove_rownames()%>%
      mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
      dplyr::select(-c(cil_1sided, vim))%>%
      dplyr::relocate(indx, landmark_time),
      col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small

```